

9319a2

COMP9319 2025T3 Assignment 2: Searching & Decoding BWT Transformed DNA Sequence (Run-length Encoded)

Your task in this assignment is to create two C/C++ programs: a search program called **bwtsearch** that implements BWT backward search, which can efficiently search a BWT transformed DNA file that has been run-length encoded; and a decoder program called **bwtdecode** that efficiently decodes such file back to its original DNA sequence. In addition to correctness, your assignment solution also needs to satisfy some performance requirements stated in this specification.

You can find some sample files available at the folder `~cs9319/a2` (you need to login to a CSE linux machine to check the content of this folder). Each of the original DNA files there (files ended with `.txt`) contains a sequences of four letters: C, G, A and T (representing four nitrogen-containing nucleobases of a nucleotide: Cytosine [C], Guanine [G], Adenine [A] or Thymine [T]) and ended with a newline (`\n`). For example:

```
cs9319@vx09:~$ cd ~cs9319/a2
cs9319@vx09:~/a2$ cat dna-tiny.txt
ACTGACTGACTGACTGAACTTACGTAGTCCAAGTA
cs9319@vx09:~/a2$
```

and its corresponding sequence transformed by BWT:

```
cs9319@vx09:~/a2$ cat dna-tiny.bwt
ATGCTGGGG
AATCTAAAAAATTTTTACAGTGGCCCCC
```

The transformed file can be easily compressed via RLE and/or MTF with a very good compression ratio, and BWT backward search can still be run against these compressed files (whereas, other usual search tools, such as `grep`, are incapable to do so). In fact, BWT is commonly used in bioinformatics to compress and index DNA data for efficient storage and search. In this assignment, you are required to search on a run-length encoded BWT file (such as those files ended with `.rbwt` in the sample file folder).

The RLE Scheme

The RLE scheme is specifically designed for this assignment. Each run of consecutive repeated characters in the BWT transformed sequence is encoded into one or more bytes. Each byte is divided into two parts:

- The first 3 bits (the most significant bits) represent the character itself.
- The remaining 5 bits (the least significant bits) represent the run length.

The character codes are assigned as follows:

- The character A is represented by the binary value 000.
- The character C is represented by 001.
- The character G is represented by 010.
- The character T is represented by 011.
- The newline character (\n) is represented by 100.

The run length is stored as a 5-bit value, which means it can represent values from 0 to 31. However, in this encoding scheme, the value 0 corresponds to a run length of 1, and the value 31 corresponds to a run length of 32. In other words, the actual run length is always one more than the value stored in the last 5 bits of the byte.

If a run of characters is longer than 32, it must be split into multiple bytes. For example, if the input contains 35 consecutive G characters, this run must be encoded using two bytes: the first byte will represent a run of 32 Gs, and the second byte will represent the remaining 3 Gs. For example, consider the following BWT transformed sequence:

AAACCCGGGGGGGGGGGGGGGGGGGGGGGGGGGGGGGGGGGGTT\n

Its run-length encoded file will contain 6 bytes as follows:

00000010 00100010 01011111 01000010 01100001 10000000

Therefore, the corresponding Run-length encoded BWT (RBWT) of the first example above becomes:

```
cs9319@vx09:~/a2$ xxd -b dna-tiny.rbwt
00000000: 00000000 01100000 01000000 00100000 01100000 01000011 . @ C
00000006: 10000000 00000001 01100000 00100000 01100000 00000110 .. .
0000000c: 01100100 00000000 00100000 00000000 01000000 01100000 d. .@`
00000012: 01000001 00100101 A%
cs9319@vx09:~/a2$
```

During the marking of your programs, only the RBWT files are provided. In the sample file folder, the original DNA files (those ended with .txt) are provided for your reference and testing only. All other files except the RBWT files will not be available during marking. Furthermore, to

easily detect any hardcoded solutions, your submitted solution may be tested with a different set of RBWT encoded DNA files.

Program Behavior

Your **bwtdecode** accepts two commandline arguments: the path to the input file (an RBWT encoded file); and the path to the output file. The input filename is a path to the given RBWT encoded file. Please open the file as read-only in case you do not have the write permission. When testing your program, we will ensure that the output file does not exist so that your program will need to create this file. For example:

```
bwtdecode ~/MyAssignmnt2/dna-tiny.rbwt myData/TinyDNA-output.result
```

You can test and check the correctness of your decoder easily by comparing your output file with the corresponding, provided source DNA files. For example:

```
cs9319@vx09: ./bwtdecode ~cs9319/a2/dna-tiny.rbwt myoutput.txt
cs9319@vx09: diff myoutput.txt ~cs9319/a2/dna-tiny.txt
cs9319@vx09:
```

Your **bwtsearch** accepts one commandline argument: the path to the input file (an RBWT encoded file). It will then take a sequence of search terms (one line each) from the standard input, performs a BWT backward search using the search term after each line of input, and output the number of matches (including zero) to the standard output. It can be input via the terminal and terminated by a Ctrl-D ("^D"). For example:

```
cs9319@vx09: ./bwtsearch ~cs9319/a2/dna-tiny.rbwt
A
12
TA
3
TAG
1
GA
5
ACT
6
ACTG
5
ACTGA
5
ACTGAC
4
```

```
AA
2
AAA
0
```

cs9319@vx09:

where the lines containing A, C, G, T are the input, and the lines containing numbers (i.e., the number of matches) are the output.

Alternatively, we can also run bwtsearch with file redirecting. For example, consider a simple sanity test that is available in the sample file folder:

cs9319@vx09: cat ~cs9319/a2/dna-tiny.input

```
A
TA
TAG
GA
ACT
ACTG
ACTGA
ACTGAC
AA
AAA
```

cs9319@vx09: cat ~cs9319/a2/dna-tiny.output

```
12
3
1
5
6
5
5
4
2
0
```

cs9319@vx09:

cs9319@vx09: ./bwtsearch ~cs9319/a2/dna-tiny.rbwt < ~cs9319/a2/dna-tiny.input >
myoutput.txt

cs9319@vx09: diff myoutput.txt ~cs9319/a2/dna-tiny.output

cs9319@vx09:

Your assignment submission will be auto-marked. Each bwtsearch on an RBWT file will contain at least 5 search terms (i.e., at least 5 lines of search input) but no more than 20 search terms,

so the runtime cost of building the backward search data structure is relatively less dominated.

A simple sanity test script is available in `~cs9319/a2`. You should at least run the sanity test script before you submit your solution. The script is very basic and just to check that the formatting of your input (such as command arguments) and output (such as newlines) is correct. In particular, marks will be deducted for output of any extra text, other than the required, correct answers. This extra information includes (but not limited to) debugging messages, line numbers and so on.

To run the sanity test script on CSE machines, simply go inside the folder that contains your makefile and program source files and type: `~cs9319/a2/autotest` that will run tests based on `dna-tiny.rbwt` provided there. It does not test on larger files provided there, as it may take much longer time for your program to complete those tests. As part of the assignment, you are expected to perform more extensive testing before submitting your solution.

To make the assignment simple, you can assume that: (1) the search is case sensitive (don't need to worry about lowercase); (2) a search term will not be an empty string; (3) a search term will not contain characters other than A, C, G and T; (4) a search term will not be longer than 100 characters.

Your solution is **not allowed** to write out any (even temporary) files, except the output file required (as specified in this document). Otherwise, you will receive zero points for the entire assignment.

Although you do not need to submit a BWT encoder, if you wish, you may implement a simple BWT encoding program (this will help you in understanding the lecture materials and assist in testing your assignment, though you are provided with sample files of different sizes at the sample file folder).

Compiling and Performance Requirements

Your solution will be compiled, run and tested on `db-perftest`. You can connect to the `db-perftest` server by firstly connecting to a CSE Linux machine (such as `vlab`) and then `ssh` to `db-perftest` using: `ssh Your_zID@db-perftest.cse.unsw.edu.au`

You may complete your assignment in C or C++. You will also need to provide a makefile to compile your source files and generate the executable programs (`bwtsearch` and `bwtdecode`) running on `db-perftest`. In case if you have never created a makefile before, a sample makefile is available in `~cs9319/a2`. You can copy and modify it accordingly for your solution. You may include and use any C or C++ libraries available on `db-perftest`. We will use the commands below to compile your solution. Please ensure that the code you submit can be compiled on `db-perftest`.

```
make
```

Any solution that has compilation errors will receive zero points for the entire assignment.

In addition to the output correctness, your solution will also be marked based on space and runtime performance. Your programs will not be tested against any DNA files that are larger than 110MB. In particular, your bwtsearch will not be tested against any RBWT encoded DNA files that are larger than 61MB, and your bwtdecode will not be tested against any RBWT encoded DNA files that are larger than 16MB. At least half of the test cases will be based on RBWT files that are smaller than 5MB (FYI, in case if you struggle with the large files).

Total runtime memory of each of the two programs (including the program footprint size) is assumed to be always less than 15MB. Runtime memory consumption will be measured by `valgrind massif` with the option `--pages-as-heap=yes`, i.e., all the memory used by your program will be measured. We will also manually check your code to ensure that your program does not try to allocate memory in a way to avoid the valgrind measurement. Any solution that violates this memory requirement will receive zero points for any relevant tests.

You may use `ms_print` to help view your massif output. For example, you can check the usage of all the memory usage snapshots of your program by reviewing the final file output from `ms_print`, i.e., `memory.txt`, as shown in the example below:

```
valgrind --tool=massif --pages-as-heap=yes --massif-out-file=memory.out ./bwtsearch ~/a2/dna-small.rbwt < mytest.in
ms_print memory.out > memory.txt
```

To measure the time performance of your solution, we will use the `time` command on `db-perftest` (i.e., `/usr/bin/time`) and take the sum of the user and system time as runtime measurement. The time limit to complete a test will be **60 seconds** for bwtdecode and **5 seconds** for each whole bwtsearch test (i.e., a test with a given BWT file and a redirecting input file that contains 5-20 search terms). Any solution that runs for more than the time limit will be killed, and will receive zero points for that test.

Documentation and Code Readability

We assume a reasonable quality of your code readability and documentation in your submitted code. Your source code will be inspected if mark scaling is applied. Marks will not be scaled up if your code is difficult to read and/or understand.

Bonus

Bonus marks (worth 10 percent of the assignment total score, i.e., 3.5 points) will be awarded for the solution that passes all the tests (correct; and within the memory and time limits) and

runs the fastest overall (i.e., the shortest total time to finish **all** the tests). Note: regardless of the bonus marks you receive in this assignment, the maximum final mark for the subject is capped at 100.

Submission

Deadline: Wednesday 12th November 12:00pm AEDT (noon).

The penalty for late submission of assignments will be 5% (of the worth of the assignment) subtracted from the raw mark per day of being late. In other words, earned marks will be lost. **No assignments will be accepted later than 5 days after the original deadline.** For example, if you have your special consideration granted by UNSW for a one-week extension, there will be no late penalty if the assignment is submitted within 7 days after the original deadline. However, no further late submissions will be accepted after these 7 days. Please read the General Assessment Information section in [the course outline](#) for details.

Use the give command below to submit the assignment or submit via WebCMS3:

```
give cs9319 a2 makefile *.c *.cpp *.h
```

Please use classrun to make sure that you have submitted the correct file(s).

```
9319 classrun -check a2
```

Note: Unfortunately the give and classrun commands are not available on db-perftest, but are on any other CSE linux machine, so you'll have to use these other machines to run the give/classrun command.

This assignment is worth 35 points, and will be auto-marked. Your code will also be checked manually for readability and plagiarism.

Plagiarism

The work you submit must be your own work. Submission of work partially or completely derived from any other person or sources is not permitted. The penalties for such an offence may include negative marks, automatic failure of the course and possibly other academic discipline. Assignment submissions will be examined both automatically and manually for such submissions.

Please read the Academic Honesty and Plagiarism section in [the course outline](#) for details.

